

Homework 2

PSTAT 131/231 Elliott Liu

Contents

Linear Regression and KNN	1
-------------------------------------	---

Linear Regression and KNN

For this assignment, we will be working with a data set from the UCI (University of California, Irvine) Machine Learning repository (see website here). The full data set consists of 4,177 observations of abalone in Tasmania. (Fun fact: Tasmania supplies about 25% of the yearly world abalone harvest.)



Figure 1: *Fig 1. Inside of an abalone shell.*

The age of an abalone is typically determined by cutting the shell open and counting the number of rings with a microscope. The purpose of this data set is to determine whether abalone age (**number of rings + 1.5**) can be accurately predicted using other, easier-to-obtain information about the abalone.

The full abalone data set is located in the `\data` subdirectory. Read it into *R* using `read_csv()`. Take a moment to read through the codebook (`abalone_codebook.txt`) and familiarize yourself with the variable definitions.

Make sure you load the `tidyverse` and `tidymodels`!

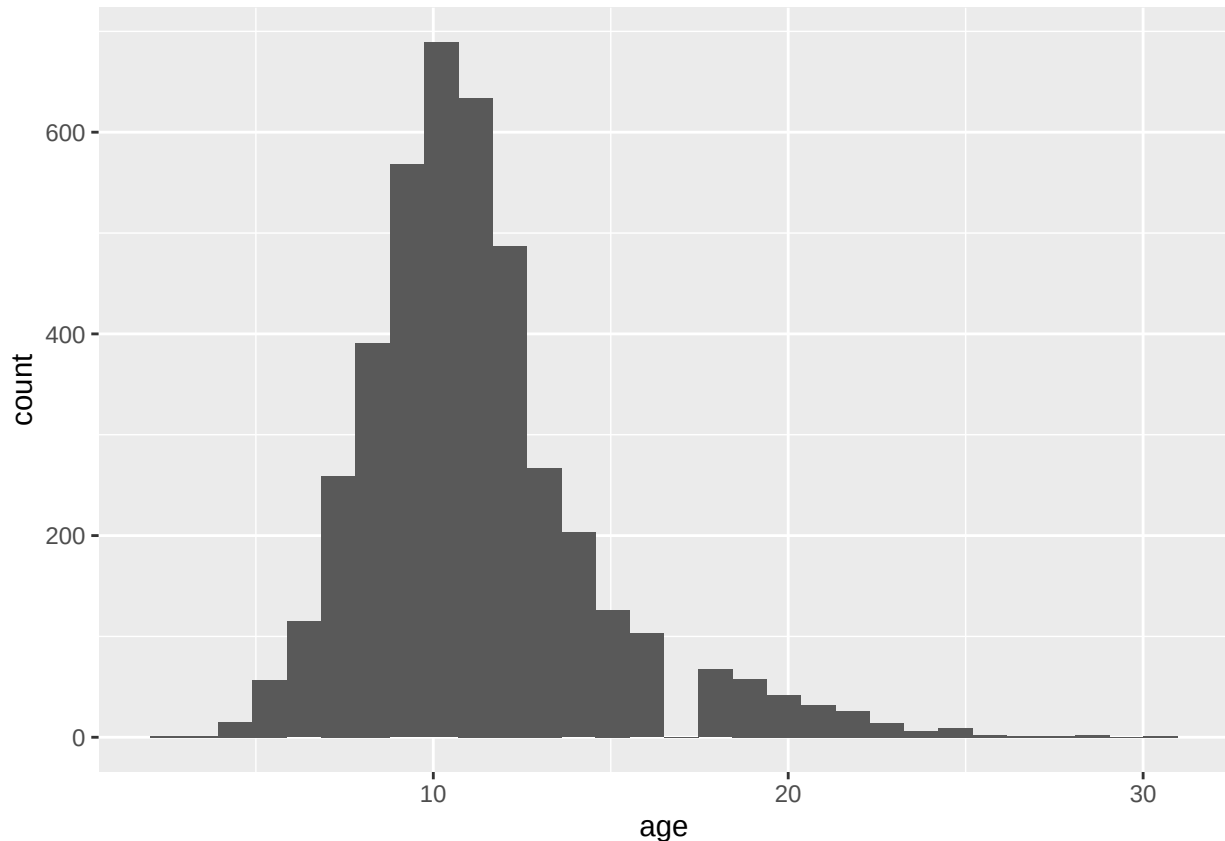
```
library(tidyverse)
library(tidymodels)
library(ggplot2)
data <- read_csv("/Users/elliottliu/Documents/PSTAT 131/hw/homework-2/data/abalone.csv")
```

Question 1

Your goal is to predict abalone age, which is calculated as the number of rings plus 1.5. Notice there currently is no `age` variable in the data set. Add `age` to the data set.

Assess and describe the distribution of `age`.

```
data2 <- data %>% mutate(age = rings + 1.5)
data2 %>% ggplot(aes(x = age)) +
  geom_histogram()
```



From the plot, we can see that after adding the `age` column, the distribution of abalone age is a unimodal and right-skewed. Most of the abalones are between 8-13 years old, and the age around 10 has the highest frequency. There's a long tail all the way to 30, which indicates that only a small number of abalones can live for a very long time.

Question 2

Split the abalone data into a training set and a testing set. Use stratified sampling. You should decide on appropriate percentages for splitting the data.

Remember that you'll need to set a seed at the beginning of the document to reproduce your results.

```
set.seed(131)
data_split <- data2 %>% initial_split(strata = age, prop = 3/4)
train <- training(data_split)
test <- testing(data_split)
```

Question 3

Using the **training** data, create a recipe predicting the outcome variable, **age**, with all other predictor variables. Note that you **should not** include **rings** to predict **age**. *Explain why you shouldn't use **rings** to predict **age**.*

Steps for your recipe:

1. dummy code any categorical predictors
2. create interactions between
 - **type** and **shucked_weight**,
 - **longest_shell** and **diameter**,
 - **shucked_weight** and **shell_weight**
3. center all predictors, and
4. scale all predictors.

You'll need to investigate the **tidymodels** documentation to find the appropriate step functions to use.

```
train_recipe <- recipe(age ~ ., data = train) %>%  
  step_rm(rings) %>%  
  step_dummy(all_nominal_predictors()) %>%  
  step_interact(  
    term = ~ starts_with("type") : shucked_weight +  
            longest_shell : diameter +  
            shucked_weight : shell_weight) %>%  
  step_normalize(all_predictors())
```

The variable **rings** should not be included when predicting the abalone age because **age** is directly derived from **rings** ($age = rings + 1.5$). Using **rings** as a predictor will result in a misleadingly high prediction accuracy due to data leakage, and the model cannot reflect true predictive performance.

Question 4

Create and store a linear regression object using the "lm" engine.

```
lm_model <- linear_reg() %>% set_engine("lm")
```

Question 5

Create and store a KNN object using the "kkn" engine. Specify **k** = 7.

```
knn_model <- nearest_neighbor(neighbors = 7) %>%  
  set_engine("kkn") %>%  
  set_mode("regression")
```

Question 6

Now, for each of these models (linear regression and KNN):

1. set up an empty workflow,
2. add the model, and
3. add the recipe that you created in Question 3.

Note that you should be setting up two separate workflows.

Fit both models to the training set.

```
lm_workflow <- workflow() %>%  
  add_recipe(train_recipe) %>%  
  add_model(lm_model)  
  
knn_workflow <- workflow() %>%  
  add_recipe(train_recipe) %>%  
  add_model(knn_model)  
  
fit_lm <- fit(lm_workflow, data = train)  
fit_knn <- fit(knn_workflow, data = train)
```

Question 7

Use your linear regression `fit()` object to predict the age of a hypothetical female abalone with `longest_shell = 0.50`, `diameter = 0.10`, `height = 0.30`, `whole_weight = 4`, `shucked_weight = 1`, `viscera_weight = 2`, and `shell_weight = 1`.

```
new_data <- tibble(  
  type = "F",  
  longest_shell = 0.50,  
  diameter = 0.10,  
  height = 0.30,  
  whole_weight = 4,  
  shucked_weight = 1,  
  viscera_weight = 2,  
  shell_weight = 1,  
  rings = 0  
)  
predict(fit_lm, new_data)
```

```
## # A tibble: 1 x 1  
##   .pred  
##   <dbl>  
## 1  23.8
```

Question 8

Now you want to assess your models' performance. To do this, use the `yardstick` package:

1. Create a metric set that includes R^2 , RMSE (root mean squared error), and MAE (mean absolute error).
2. Use `augment()` to create a tibble of your model's predicted values from the **testing data** along with the actual observed ages (these are needed to assess your model's performance).
3. Finally, apply your metric set to the tibble, report the results, and interpret the R^2 value.

Repeat these steps once for the linear regression model and for the KNN model.

```
metric <- yardstick::metric_set(rmse, rsq, mae)
testing_results_lm <- augment(fit_lm, test)
metric(
  testing_results_lm,
  truth = age,
  estimate = .pred
)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 rmse    standard      2.14
## 2 rsq     standard      0.556
## 3 mae     standard      1.55
```

```
testing_results_knn <- augment(fit_knn, test)
metric(
  testing_results_knn,
  truth = age,
  estimate = .pred
)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>      <dbl>
## 1 rmse    standard      2.35
## 2 rsq     standard      0.478
## 3 mae     standard      1.69
```

On the testing data, the linear regression model has an R^2 of 0.5561215, RMSE of 2.1430158, and MAE of 1.5531388, indicating that on average its predictions deviate from the true age by approximately 1.55 years. However, KNN model has an R^2 of 0.4780108, RMSE of 2.3487667, and MAE of 1.6902146.

R^2 represents the proportion of the variance in the response variable explained by the model, and a higher R^2 means a better model fit to the data. In the linear regression model, R^2 is 0.5561215, meaning that this model explains 55.61% of the variability in abalone age. However, the R^2 of the KNN model is 0.4780108, which is smaller than that in the linear regression model. Therefore, the linear regression model has a better predictive performance than the KNN model.

Question 9

Which model performed better on the testing data? Explain why you think this might be. Are you surprised by any of your results? Why or why not?

On the testing data, the linear regression model performed better than the KNN model, due to its higher R^2 and lower RMSE and MAE values. This means that the linear regression model explains more variability in abalone age and produces more accurate predictions.

This result is reasonable because the abalone age and the predictor variables change gradually and can be approximated by a linear structure. In contrast, KNN tends to perform better when the data has strong patterns or clear clusters; however, the data in this dataset did not show clear clusters or strong patterns. Therefore, I am not surprised by these results, because linear regression performs well for gradually changing relationships.

Required for 231 Students

In lecture, we presented the general bias-variance tradeoff, which takes the form:

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon)$$

where the underlying model $Y = f(X) + \epsilon$ satisfies the following:

- ϵ is a zero-mean random noise term and X is non-random (all randomness in Y comes from ϵ);
- (x_0, y_0) represents a test observation, independent of the training set, drawn from the same model;
- $\hat{f}(\cdot)$ is the estimate of f obtained from the training set.

Question 10 Which term(s) in the bias-variance tradeoff above represent the reducible error? Which term(s) represent the irreducible error?

Question 11 Using the bias-variance tradeoff above, demonstrate that the expected test error is always at least as large as the irreducible error.

Question 12 Prove the bias-variance tradeoff.

Hints:

- use the definition of $\text{Bias}(\hat{f}(x_0)) = E[\hat{f}(x_0)] - f(x_0)$;
- reorganize terms in the expected test error by adding and subtracting $E[\hat{f}(x_0)]$